Universidade do Minho
Licenciatura em Engenharia Informática

Sistemas Distribuídos

Trabalho Prático

Grupo 30

Gonçalo Amaro
a106803

Luís Maciel
a106896

Octávio Henriques
a104277

1. Introdução

O objetivo deste projeto foi o desenvolvimento de um sistema cliente-servidor para a gestão de grandes volumes de vendas e análise de dados temporais. O sistema foi desenhado para suportar acesso concorrente, persistência de dados e um protocolo de comunicação binário eficiente sobre TCP.

Os principais desafios abordados incluíram a gestão eficiente de memória no servidor (para suportar históricos longos), a implementação de um cliente multi-threaded capaz de multiplexar pedidos numa única conexão TCP, e o suporte a operações bloqueantes (notificações) sem comprometer a disponibilidade do sistema.

2. Explicação

A solução baseia-se num protocolo de troca de mensagens customizado, onde cada trama (frame) possui um cabeçalho composto por Tamanho, Tag (Identificador do Pedido) e OpCode (Tipo de Operação).

2.1. Serialização

Para otimizar a largura de banda nas operações de listagem(GET_EVENTS), em vez de enviar Strings repetidas (ex: “Batata” , “Batata”, “Batata”), o servidor envia primeiro um dicionário de produtos e, em seguida, uma lista de índices que referenciam esse dicionário. Do lado do cliente, a “ClientLib” reconstrói os dados originais, reduzindo significativamente o tamanho da resposta.

2.2. Modelo de Concorrência

A arquitetura do servidor foi desenhada para resolver o clássico "Problema dos Leitores-Escritores". Analisando o padrão de carga do sistema, verifica-se uma assimetria clara: existem frequentes operações de consulta (leituras de agregações, médias, volumes) e raras operações de modificação estrutural (apenas na mudança de dia e inserção de vendas).

Para maximizar o débito (*throughput*) do sistema, aplicou-se um ReentrantReadWriteLock no motor de séries temporais. Esta primitiva permite que múltiplas threads de leitura acedam às estruturas de dados simultaneamente (paralelismo real na consulta), bloqueando apenas quando a thread de escrita necessita de exclusividade para persistir e rodar o dia atual.

Adicionalmente, na implementação das funcionalidades de subscrição (wait/notify), utilizou-se a primitiva Condition. Um detalhe crítico foi a proteção contra "Spurious Wakeups" (acordares espontâneos). As threads que aguardam por vendas fazem-no sempre dentro de um ciclo de verificação (while (!condicao) { await(); }). Isto garante que, mesmo que o sistema operativo acorde a thread indevidamente, o estado é reavaliado atomicamente antes de prosseguir.

2.3. Gestão de Memória (LRU)

Dado o requisito de suportar séries temporais longas com memória limitada (S dias), implementou-se um gestor de histórico (HistoryManager) que atua como uma cache entre a RAM e o Disco.

A política de substituição de páginas escolhida foi a LRU (Least Recently Used). Foi utilizada uma estrutura de fila para rastrear a ordem de acesso aos dias. Sempre que um dia é consultado, este é movido para o fim da fila, ganhando prioridade de permanência. Quando o limite de memória é atingido, o dia no início da fila (o menos recentemente usado) é o candidato escolhido para ser removido da memória e libertar recursos.

2.4. Otimização "Lazy" nas Agregações

Para otimizar o processamento de grandes volumes de dados, adotou-se uma estratégia de Lazy Evaluation. Os cálculos pesados (como médias globais e volumes acumulados) não são realizados no momento da inserção da venda. Apenas quando um cliente solicita a informação é que o cálculo é efetuado e o resultado armazenado numa cache interna (cacheValues). Pedidos subsequentes para o mesmo produto devolvem o valor em cache instantaneamente, reduzindo drasticamente a carga de CPU em produtos populares.

3. Arquitetura

3.1. Arquitetura de cliente

O cliente utiliza uma arquitetura baseada no padrão “Demultiplexer”.

- Uma única thread de leitura (“readLoop”) recebe todas as respostas do socket e coloca-as num Mapa (“responses”), sinalizando a thread que está à espera daquela Tag especifica.

- Isto permite que várias threads de aplicação utilizem a mesma conexão TCP simultaneamente sem bloquear umas às outras, utilizando variáveis de condição para sincronização.

3.2. Arquitetura do servidor

O servidor segue o modelo “Thread per Client”, onde cada conexão é gerida por um “Session”. A lógica de negócio está centralizada no “TimeSeriesEngine”, dividida em dois componentes:

- **CurrentDay:** Gere as vendas do dia atual na RAM, utilizando “ReentrantLock” e “Condition” para gerir concorrência e notificações em tempo real.

- **HistoryManager:** Gere a persistência e o carregamento de dias passados do disco, implementado a lógica de gestão de cache e transient fields para otimizar a serialização Java.

4. Testes

Para validar as decisões de desenho, foram realizados testes focados em carga, notificações e robustez.

4.1 Cenário de Carga e Escalabilidade

Utilizou-se o “LoadTester” para simular um ambiente de alta concorrência com 10 clientes “Normais” (operações de leitura e escrita) e 2 clientes “Waiters” (notificações).

- **Resultados:** O sistema processou 2000 operações em aproximadamente 591 ms, resultando num throughput elevado (~3384 ops / sec) sem erros de conexão ou dados corrompidos.
- **Thread-safety:** O teste confirmou que o “demultiplexer” encaminha corretamente as respostas mesmo sob carga intensa.

```
>>> INICIANDO TESTE DE CARGA <<<
Clientes Normais: 10
Ops por Cliente: 200
(Waiter 1) À espera de 50 Batatas consecutivas...
(Waiter 0) À espera de 50 Batatas consecutivas...
Cliente 0 terminou.
Cliente 9 terminou.
Cliente 1 terminou.
Cliente 2 terminou.
Cliente 4 terminou.
Cliente 5 terminou.
Cliente 3 terminou.
Cliente 6 terminou.
Cliente 7 terminou.
Cliente 8 terminou.

=== RESULTADOS ===
Tempo Total: 591ms
Sucessos: 2000
Falhas: 0
Throughput Estimado: 3384.0947546531306 ops/sec
```

Figura 1: Output do teste LoadTester.

4.2. Notificações Bloqueantes

O teste “ScenarioTest” e o “TesteNotificacao” validaram a lógica de eventos.

- Um cliente “Observador” subscrever vendas consecutivas e ficou bloqueado no servidor (“await()”) sem consumir CPU (busy waiting).
- Assim que o “Vendedor” atingiu o critério, o “CurrentDay” sinalizou a condição (“change.signalAll()”), desbloquando o observador imediatamente.

```
>>> INICIANDO TESTE DE NOTIFICACAO <<<
[OBS] Subscrevendo 3 vendas consecutivas de 'ProdutoX'...
[VEND] A vender ProdutoX (1/3)...
[VEND] A vender ProdutoX (2/3)...
[VEND] A vender ProdutoX (3/3)...
[OBS] !!! Notificacao Recebida !!!
[OBS] Mensagem: 3 vendas consecutivas de ProdutoX!
[VEND] Vendas concluidas.
```

Figura 2: Output do teste TesteNotificacao.

```
>>> INICIANDO TESTE DE CENÁRIO (NOTIFICAÇÕES) <<<
[OBSERVADOR] Vou subscrever '50 Batatas' e esperar...
[VENDEDOR] Vou vender 20 Batatas...
[VENDEDOR] Vou vender mais 20 Batatas (Total 40)...
[VENDEDOR] Vou vender mais 15 Batatas (Total 55)...
[VENDEDOR] Trabalho concluído.

[OBSERVADOR] !!! ACORDEI !!! Mensagem: 50 vendas consecutivas de Batata!
>>> CENÁRIO CONCLUÍDO COM SUCESSO <<<
```

Figura 3: Output do teste ScenarioTest.

4.3. Persistência e Recuperação

Através do “ServerMain”, verificou-se a capacidade de recuperação de falhas. O uso de um “Shutdown Hook” garante que, ao terminar o servidor, o estado atual é gravado em disco de forma segura. A leitura do estado atual (“carregarEstadoDia”) permite retornar a operação sem perca de dados.

```
Iniciando Povoamento de Dados
User [utilizador_2] iniciou a injeção de dados...
User [utilizador_1] iniciou a injeção de dados...
User [utilizador_5] iniciou a injeção de dados...
User [utilizador_4] iniciou a injeção de dados...
User [utilizador_3] iniciou a injeção de dados...
User [utilizador_4] terminou.
User [utilizador_1] terminou.
User [utilizador_3] terminou.
User [utilizador_2] terminou.
User [utilizador_5] terminou.

 -> CONCLUIDO
Total de vendas inseridas: 500
Tempo decorrido: 84ms
```

Figura 4: Output do teste PovoamentoTest.

5. Conclusão

O sistema desenvolvido cumpre integralmente os objetivos propostos, apresentando uma arquitetura robusta e escalável. No servidor, a adoção do padrão Leitores-Escritores, implementado através de ReentrantReadWriteLock, permitiu maximizar o paralelismo nas operações de consulta, evitando gargalos de desempenho e garantindo simultaneamente a consistência dos dados nas operações de escrita.

Relativamente à gestão de recursos, a solução demonstrou eficiência no tratamento de grandes volumes de dados. A implementação de uma política de substituição LRU (Least Recently Used) assegurou uma gestão transparente da hierarquia entre a memória RAM limitada e o armazenamento em disco, enquanto a estratégia de *Lazy Evaluation* otimizou o uso de CPU nas agregações estatísticas.

Por fim, ao nível da comunicação, o protocolo binário customizado garantiu uma baixa latência e uso reduzido de largura de banda. A arquitetura de Multiplexagem no cliente validou com sucesso o suporte a operações multi-threaded concorrentes sobre uma única conexão TCP, resultando numa solução final resiliente e preparada para cenários de carga elevada.